

Error Code Property Proposal

A TC39 Proposal

Stage 2 / 2.7?

James M Snell

The Problem

How do you identify a specific error condition today?

```
try {
  await fetch(url);
} catch (e) {
  // .message? Fragile -- varies across engines and versions
  // (I broke npm once by adding a period to an error message)
  if (e.message.includes('network')) { /* ... */ }

  // instanceof? Breaks across realms and structured cloning
  if (e instanceof TypeError) { /* ... */ }

  // .name? Too coarse -- every TypeError shares the same name
  if (e.name === 'TypeError') { /* ... */ }
}
```

`.name` tells you *what kind* of error (one of ~7 built-in types), but not *which specific* condition. All `TypeError`s look the same.

Even `DOMException`, which repurposes `.name` for specific identity (`"AbortError"` , `"NotFoundError"`), demonstrates the problem - `instanceof` and `.name` disagree, stack traces are misleading.

The Proposal

Extend the `Error` constructor options bag to accept `code`

```
new Error("something went wrong", { code: "ERR_SOMETHING" })
```

```
new TypeError("expected string", {  
  code: "ERR_INVALID_ARG_TYPE",  
  cause: original  
})
```

```
try {  
  // do something that fails  
  throw new Error("Connection refused", { code: "ERR_CONNECTION_REFUSED" });  
} catch (e) {  
  if (e.code === 'ERR_CONNECTION_REFUSED') {  
    // Retry with backoff  
  }  
}
```

Design

Mirrors the `cause` property exactly

Defined on	Instances, not <code>Error.prototype</code>
Type	Any value (consistent with <code>cause</code>)
Default	Absent when not provided (<code>'code' in err</code> is <code>false</code>)
Enumerable	<code>false</code>
Writable	<code>true</code>
Configurable	<code>true</code>

Why Do We Need This?

Seven motivations

1. **Precise programmatic error handling** – branch on codes, not message strings or names, which are either non-specific or reproduce DOMException-like issues
2. **Stable contracts across versions** – codes are documented, versioned API
3. **Cross-realm error identification** – survives structured cloning, `postMessage`
4. **Improved telemetry** – aggregate by code in Sentry, Datadog, etc.
5. **i18n support** – stable codes enable localized error messages
6. **Ecosystem alignment** – bless the pattern everyone already uses
7. **Better DX** – searchable, documentable error identifiers

Prior Art: Runtimes

Every major JS runtime already does this

Node.js (since v8.0, 2017)

- 200+ documented `ERR_*` string codes
- POSIX codes for system errors
- Part of the stable API

Deno – mirrors Node.js codes in compat layer

Bun – mirrors Node.js codes in compat layer

Cloudflare Workers – mirrors Node.js codes

```
// Node.js
const fs = require('fs');
try {
  fs.readFileSync('/nonexistent');
} catch (e) {
  e.code;    // 'ENOENT'
  e.message; // "ENOENT: no such ..."
}
```

```
// axios
try {
  await axios.get(url);
} catch (e) {
  e.code; // 'ERR_NETWORK'
}
```

```
// AWS SDK v3
try {
  await s3.getObject(params);
} catch (e) {
  e.code; // 'NoSuchBucket'
}
```

Prior Art: Libraries

The ecosystem has already converged on `.code`

Library	<code>.code</code> type	Convention
axios	<code>string</code>	<code>ERR_NETWORK</code> , <code>ERR_CANCELED</code>
Firebase	<code>string</code>	<code>"auth/user-not-found"</code>
Stripe	<code>string</code>	<code>"card_declined"</code> , <code>"rate_limit"</code>
Prisma	<code>string</code>	<code>"P2002"</code> , <code>"P2025"</code>
pg (Postgres)	<code>string</code>	SQLSTATE: <code>"23505"</code> , <code>"42P01"</code>
mysql2	<code>string</code>	<code>"ER_DUP_ENTRY"</code>
AWS SDK v3	<code>string</code>	<code>"AccessDenied"</code> , <code>"NoSuchBucket"</code>

Prior Art: Web Platform

The web already learned this lesson

DOMException - what happens when `.name` is overloaded for error identity:

- `err instanceof DOMException` is `true`, but `err.name` is `"AbortError"` - not `"DOMException"`
- `.name` became a de facto error code while still nominally being the "type name"
- Stack traces show `"AbortError"` - looks like a class that doesn't exist
- Difficult to subclass `.name` without losing error identity

Had `.code` existed, `DOMException` could have used `{ code: "AbortError" }` while keeping `.name` as `"DOMException"`.

Broader trend: Older Web APIs used numeric codes (`MediaError`, `GeolocationPositionError`). Newer ones use strings (`RTCErrors` `.errorDetail`). The platform has moved toward string-based identification.

DOMException Interaction

DOMException has `Error` in its prototype chain and a legacy numeric `.code` – is there a conflict?

- `code` is only installed when passed via the constructor options bag – it is never set by default
- DOMException's constructor does not accept an options bag, so `InstallErrorOwnProperties` is never invoked – no shadowing occurs
- This proposal does **not** modify `DOMException` – not web-breaking
- A single web platform type's legacy use of a property name should not constrain language-level improvements

The `.code` property name overlap already exists today – Node.js, Deno, Bun, and hundreds of libraries all set `.code` on Error instances alongside DOMException in the same runtimes. The ecosystem has already made this choice.

Spec Changes

Replaces `InstallErrorCause` with a consolidated `InstallErrorOwnProperties`

```
InstallErrorOwnProperties ( O, options )
1. If options is an Object, then
  a. If ? HasProperty(options, "cause") is true, then
    i. Let cause be ? Get(options, "cause").
    ii. Perform CreateNonEnumerableDataPropertyOrThrow(
        O, "cause", cause).
  b. If ? HasProperty(options, "code") is true, then
    i. Let code be ? Get(options, "code").
    ii. Perform CreateNonEnumerableDataPropertyOrThrow(
        O, "code", code).
2. Return unused.
```

Replaces `InstallErrorCause` in: `Error()`, `NativeError()`, `AggregateError()`, `SuppressedError()`

Relationship to Other Proposals

This builds on and complements existing work

Proposal	Relationship
Error.cause (ES2022)	Established the options bag pattern. This extends it.
Error Stacks (Stage 1)	Orthogonal – stacks could include codes.
Error.isError (Stage 2)	Complementary – <code>.code</code> provides fine-grained identification within a confirmed error.

FAQ

Common questions

"Won't every library invent its own codes?" They already do. Standardizing the *property* doesn't require standardizing the *values*. It gives a blessed location instead of ad-hoc `.errno` , `.errorCode` , `.errCode` .

"Why not use Symbols?" Symbols prevent collisions but sacrifice serialization, logging, telemetry, and cross-realm transfer. But `code` accepts any value – use Symbols if you prefer.

"Can't you solve this with subclasses?" `instanceof` breaks across realms. The built-in hierarchy is too shallow. Structured cloning doesn't preserve classes. The ecosystem has voted with its feet.

Thank You

code Property for Error Objects

Asking for Stage 2?

How about Stage 2.7?

Proposal | Spec Text